

13 组-实验五

赵心瑞

林佳慧

陈莹

24307140096

24307140111

24300240197

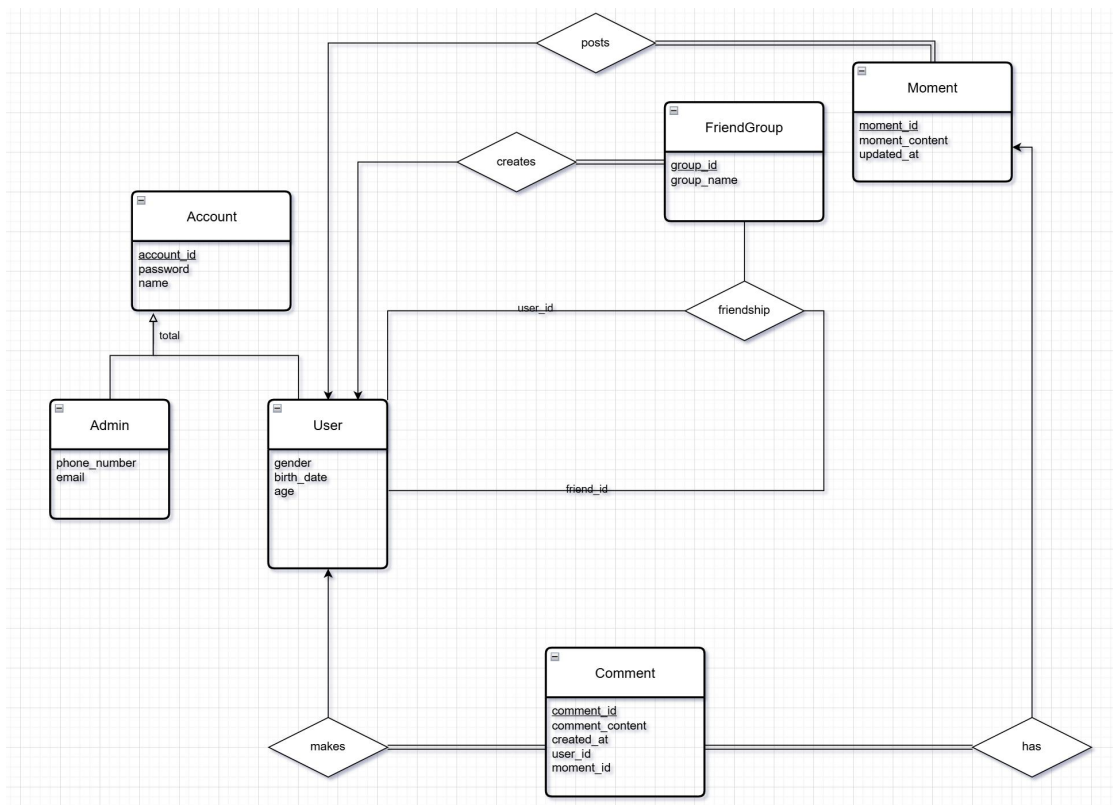
一. 系统总体设计

我们实现了一个简化版社交平台 Mini-WeChat，使用 Python 作为开发语言，MySQL 作为数据库。该系统同时提供命令行入口和基于 Streamlit 的 Web 页面，两种交互方式共用同一套后端业务逻辑和数据库访问代码。

系统分为普通用户端和管理员端。普通用户可以注册、登录，修改个人信息，管理好友和好友分组，发布、修改、删除自己的朋友圈，查看好友朋友圈，并对自己或好友的朋友圈发表评论。管理员可以登录，修改个人信息，查看全站朋友圈及评论，删除违规朋友圈，也可以注销用户账号，并通过外键级联机制清理该用户相关数据。

代码结构上，db_config.py 负责数据库连接和角色设置，auth.py 负责注册与登录，user_controller.py 负责普通用户业务，admin_controller.py 负责管理员业务，handle_error.py 负责数据库错误处理。main.py 是命令行入口，web_main.py 是 Streamlit 前端入口。数据库名为 Mini_WeChat，核心表包括 User、Admin、FriendGroup、Friendship、Moment 和 Comment。

二. ER 图



三. 表结构与约束

各表的字段、数据类型及约束条件设计如下所示。

User (user_id, password, name, gender, birth_date, age), 其中 user_id 是主键, password、name 非空, gender 有 check 约束 (在 male 和 female 中选)。

Admin (admin_id, password, name, phone_number, email), 其中 admin_id 是主键, password、name、phone_number 非空。

FriendGroup (group_id, group_name, created_by), 其中 group_id 是主键并且是 auto_increment 的, group_name 非空, created_by 有外键约束、参照 User 表的 user_id 并且和它级联删除。

Friendship (user_id, friend_id, group_id), 其中 (user_id, friend_id) 是联合主键, user_id 有外键约束、参照 User 表的 user_id 并且和它级联删除, friend_id 有外键约束、参照 User 表的 user_id 并且和它级联删除, group_id 默认为 null, 有外键约束、参照 FriendGroup 表的 group_id 并且 on delete set null, 即在该好友分组被删除时, 朋友关系不会被删除, 只是回到无分组状态。

Moment (moment_id, moment_content, user_id, updated_at), 其中 moment_id 是主键并且是 auto_increment 的, moment_content 为 VARCHAR(600), 不能为空, 长度最多 600 个字符, user_id 有外键约束、参照 User 表的 user_id 并且和它级联删除, updated_at 是 datetime 类型 default current_timestamp 并且自动更新为最新的修改时间 on update current_timestamp。

Comment (comment_id, comment_content, user_id, moment_id, created_at), 其中 comment_id 是主键并且是 auto_increment 的, comment_content 为 VARCHAR(200), 不能为空, 长度最多 200 个字符, user_id 有外键约束、参照 User 表的 user_id 并且和它级联删除, moment_id 有外键约束、参照 Moment 表的 moment_id 并且和它级联删除, created_at 是 datetime 类型、默认值为当前时间。

user_id、admin_id、friend_id、created_by 的数据类型均为 VARCHAR(50), birth_date 为 DATE 类型, age 为 INT 类型, password 为 VARCHAR(20), phone_number 是 VARCHAR(11), email 是 VARCHAR(20), group_id、moment_id、comment_id 为 INT 类型, group_name、name 为 VARCHAR(40), gender 为 VARCHAR(6), created_at、updated_at 是 datetime 类型, moment_content 是 VARCHAR(600), comment_content 是 VARCHAR(200)。

系统主要通过主键约束、非空约束、CHECK 约束、外键约束、字段长度限制等保证数据完整性。业务层还补充了应用级约束, 例如用户不能添加或删除自己, 用户只能修改或删除自己的朋友圈, 用户只能评论自己或好友的朋友圈, 移动好友到分组前会检查该分组是否属于当前用户。

四、事务与触发器

系统最主要的事务用于维护双向好友关系。add_friend 函数会开启事务, 连续向 Friendship 表插入 A 到 B、B 到 A 两条记录, 全部成功后提交; 若发生主键冲突、外键错误

或其他异常，则回滚，避免只建立单向好友关系。`delete_friend` 函数同样使用事务，同时删除两个方向的好友关系，保证删除操作的一致性。

其他写操作大多是单条 INSERT、UPDATE 或 DELETE，执行成功后提交。管理员删除用户时，应用层只删除 User 表记录，相关好友关系、好友分组、朋友圈和评论由数据库外键级联自动清理。

系统定义了两个触发器 `Get_age_insert` 和 `Get_age_update`。二者分别在 User 表插入和更新前执行，通过 `TIMESTAMPDIFF(YEAR, NEW.birth_date, CURDATE())` 计算年龄，并写入 `NEW.age`。这样用户注册或修改出生日期时，age 字段会由数据库自动维护。

五、视图、权限与错误处理

我们的系统创建了 `Admin_Moment_View` 视图供管理员审核朋友圈。该视图由 Moment 左连接 Comment 得到，包含 `moment_id`、`moment_content`、`user_id`、`updated_at` 和 `comment_content` 等信息。使用 LEFT JOIN 可以保证没有评论的朋友圈也能被管理员看到。

系统创建了 `Role_user` 和 `Role_admin` 两个数据库角色。`Role_user` 拥有对 User 表的 SELECT、UPDATE 权限，以及对 Friendship、FriendGroup、Moment、Comment 表的增删改查权限。`Role_admin` 拥有对 Admin 表的 SELECT、UPDATE 权限，对 `Admin_Moment_View` 的 SELECT 权限，以及对 Moment 表和 User 表的 DELETE 权限。数据库角色控制的是表级权限，至于用户只能操作本人数据等行级限制则是由后端 SQL 的 WHERE 条件实现的。

系统在 `handle_error.py` 集中处理常见数据库错误，包括重复主键、字段超长、CHECK 约束失败、非空约束失败和外键引用失败等，并将其转换为用户可理解的提示，例如“Publish moment failed: Moment content exceeds the maximum limit of 600 characters.”、“Publish comment failed: Comment exceeds the maximum limit of 200 characters.”和“Add friend failed: Your friend is already added.”等。业务层也会提前检查自加好友、目标朋友圈不存在、评论权限不足和评论为空等情况。

六. 小组分工

代码实现：24307140096 赵心瑞

数据库设计、报告撰写：24307140111 林佳慧

系统测试、报告撰写：24300240197 陈莹